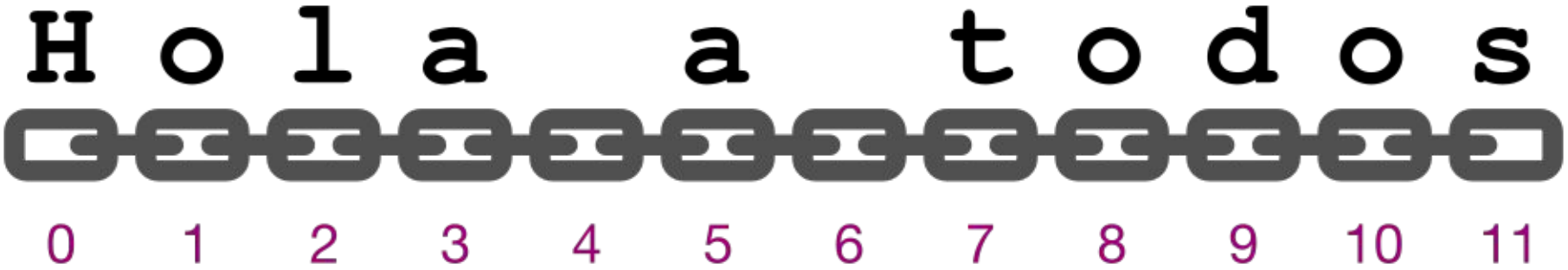
# Strings

# Cadenas (strings)

una cadena de caracteres o cadena de texto o simplemente cadena (string en inglés) es una secuencia ordenada de símbolos, con una longitud arbitraria (con tantos símbolos como queramos).

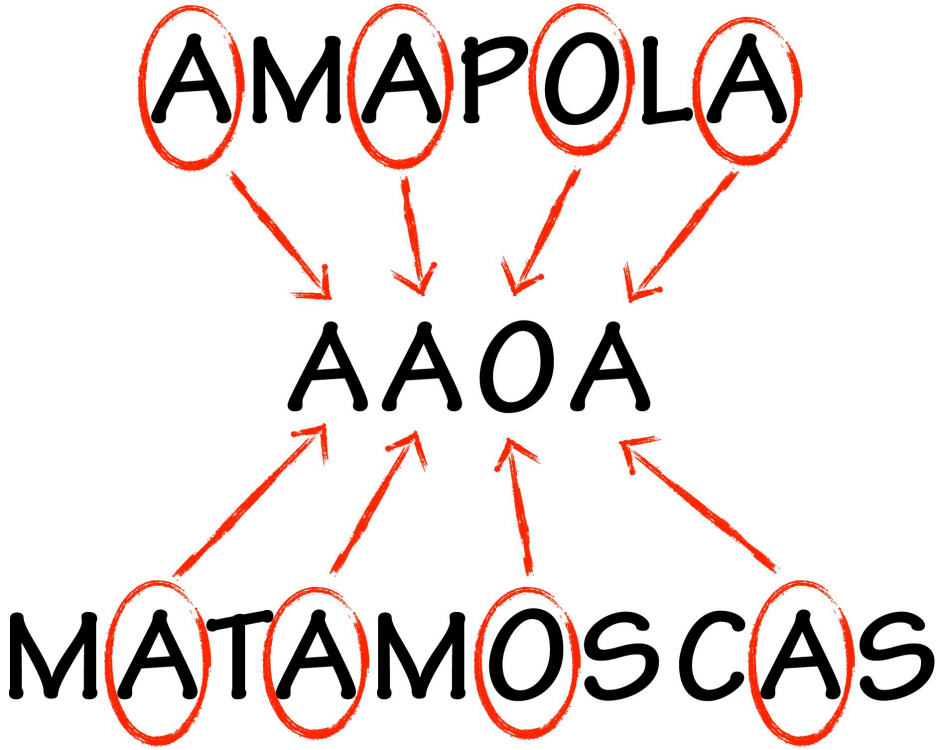


# Que simbolos que utilizamos?

```
cook@pop-os:~$ ascii -d
    0 NUL    16 DLE    32       48 0    64 @    80 P    96 `   112 p
    1 SOH    17 DC1    33 !     49 1    65 A    81 Q    97 a   113 q
    2 STX    18 DC2    34 "     50 2    66 B    82 R    98 b   114 r
    3 ETX    19 DC3    35 #     51 3    67 C    83 S    99 c   115 s
    4 EOT    20 DC4    36 $     52 4    68 D    84 T   100 d   116 t
    5 ENQ    21 NAK    37 %     53 5    69 E    85 U   101 e   117 u
    6 ACK    22 SYN    38 &     54 6    70 F    86 V   102 f   118 v
    7 BEL    23 ETB    39 '     55 7    71 G    87 W   103 g   119 w
    8 BS     24 CAN    40 (     56 8    72 H    88 X   104 h   120 x
    9 HT     25 EM     41 )     57 9    73 I    89 Y   105 i   121 y
   10 LF     26 SUB    42 *     58 :    74 J    90 Z   106 j   122 z
   11 VT     27 ESC    43 +     59 ;    75 K    91 [   107 k   123 {
   12 FF     28 FS     44 ,     60 <    76 L    92 \   108 l   124 |
   13 CR     29 GS     45 -     61 =    77 M    93 ]   109 m   125 }
   14 SO     30 RS     46 .     62 >    78 N    94 ^   110 n   126 ~
   15 SI     31 US     47 /     63 ?    79 O    95 _   111 o   127 DEL
```

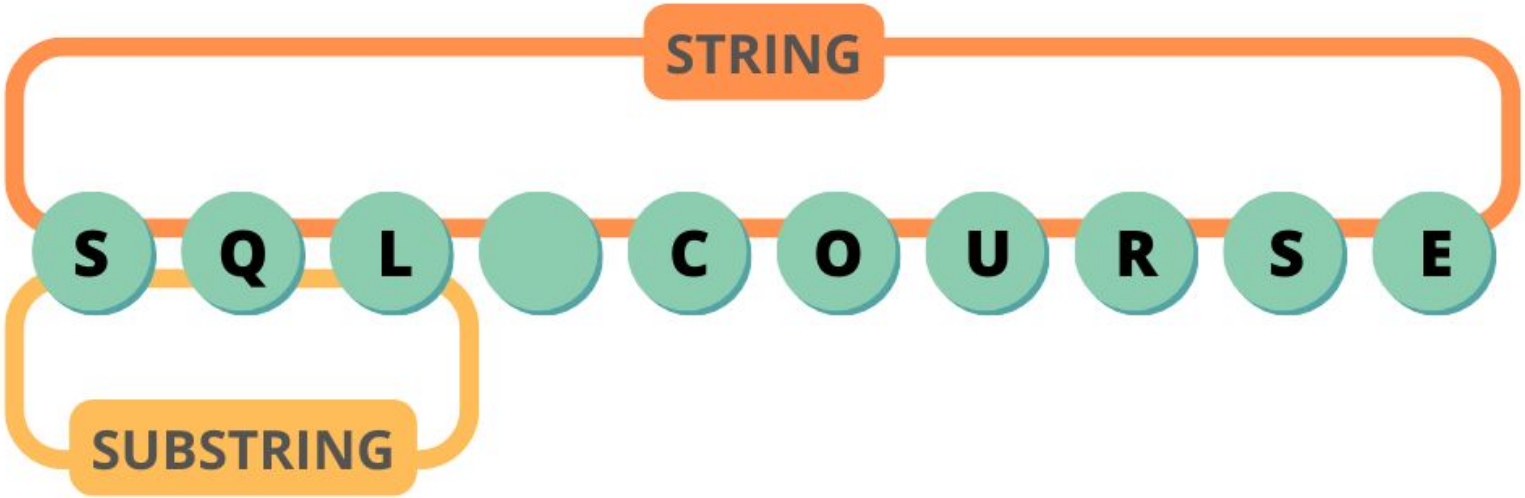
# Subsecuencia

una subsecuencia C es un subconjunto ordenado de caracteres de la secuencia A no necesariamente consecutivos



# Subcadena o substring

Las subcadenas son caracteres secuenciales dentro de una cadena. Por ejemplo, la cadena "abc" tiene las siguientes subcadenas: "" , "a" , "ab" , "abc" , "b" , "bc" , "c" .



# Problema principal: coincidencia de cadenas o busqueda

Consiste en encontrar la(s) ocurrencia(s) de un texto (patron de busqueda) en otro texto mas largo(dominio de busqueda), ademas de determinar si es que dicho patron no existe en mi dominio de busqueda.

# Problema principal: coincidencia de cadenas o busqueda

1001110100101000101001110001 11

Reconocer la cadena: 10100111

```
1001110100101000101001110001 11
10100111
 10100111
  10100111
   10100111
    10100111
     10100111
      10100111
1001110100101000101001110001 11
```

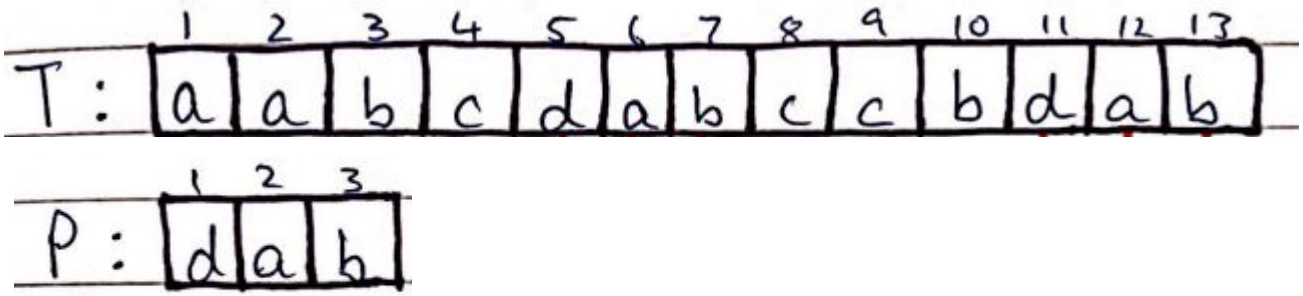
# Soluciones

- Fuerza bruta
- KMP
- Algoritmo Z
- Rabin Karp

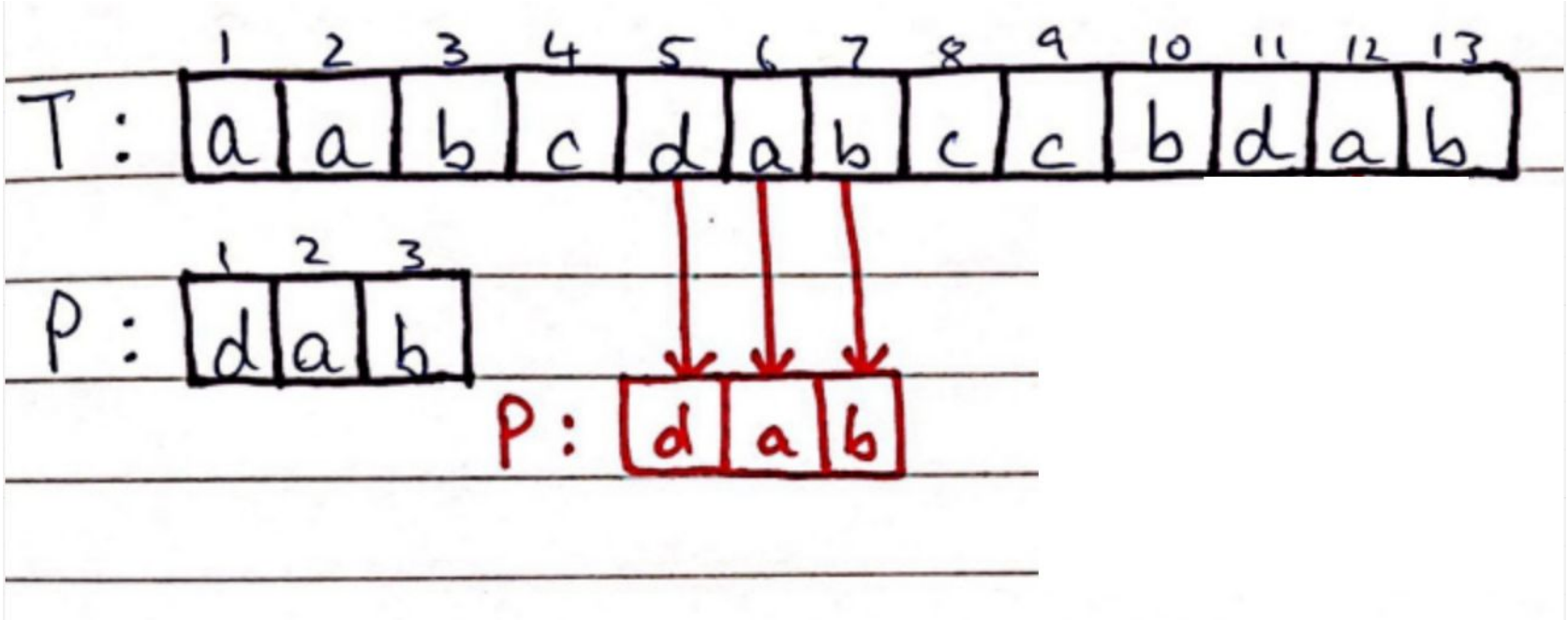
```
public class Pattern {
  public static void main(String[] args) {
      Pattern obj=new Pattern();
      obj.pattern("SCHOOL");
  }
  void pattern(String s) {
     for(int i=0;i<s.length();i++)
       System.out.println(s.substring(0,
i+}));
}
```

# Solucion por fuerza bruta

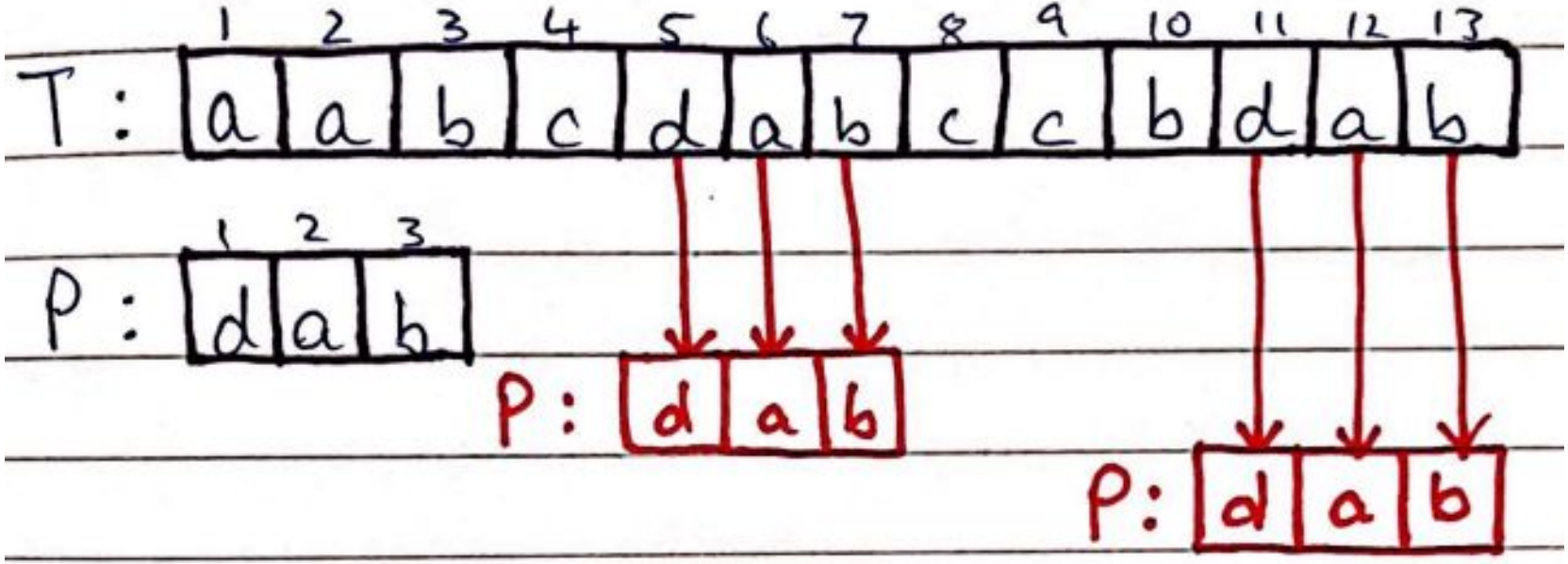
Supongamos que tenemos las siguientes cadenas:



# Solucion por fuerza bruta



# Solucion por fuerza bruta



# Solucion por fuerza bruta

**do**
  **if** (text letter == pattern letter)
    compare next letter of pattern to next letter of text
  **else**
    move pattern down text by one letter
**while** (entire pattern found or end of text)

```
#define vector<int> vi
vi brute(string &t, string &p) {
    vi res;
    for (int i=0 ; i<t.size(); i++) {
        string k = t.substr(i, p.size());
        if (k==p) res.pb(i);
    }
    return res;
}
```

# Como se puede optimizar el algoritmo de fuerza bruta?

# Solucion KMP

El algoritmo KMP es un algoritmo de búsqueda de subcadenas simple y por lo tanto su objetivo es buscar la existencia de una subcadena dentro de una cadena. Para ello utiliza información basada en los fallos previos, aprovechando la información que la propia palabra a buscar contiene de sí (sobre ella se precalcula una tabla de valores), para determinar donde podría darse la siguiente existencia, sin necesidad de analizar más de 1 vez los caracteres de la cadena donde se busca.

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T : | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | 1 | 2 | 3 |
|---|---|---|---|
| P : | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | | | 1 | | | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | | | | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | | | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | | | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | 1 | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | 1 | 2 | | | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | 1 | 2 | 3 | | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

MATCH!!!

# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

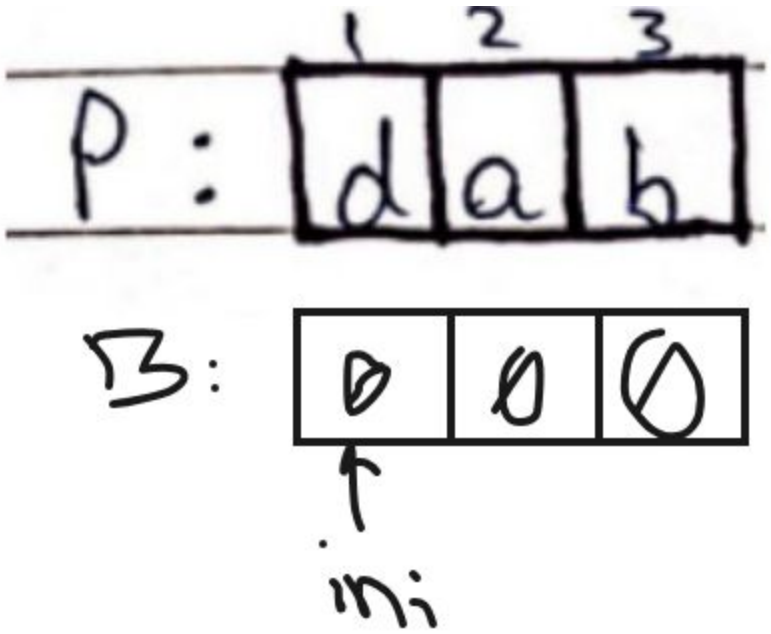
| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | 1 | 2 | 3 | ? | | | | |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

NO SON IGUALES, POR LO TANTO DESDE DONDE DEBO SEGUIR??

# Solucion KMP

# Solucion KMP



# Solucion KMP

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| T: | a | a | b | c | d | a | b | c | c | b | d | a | b |

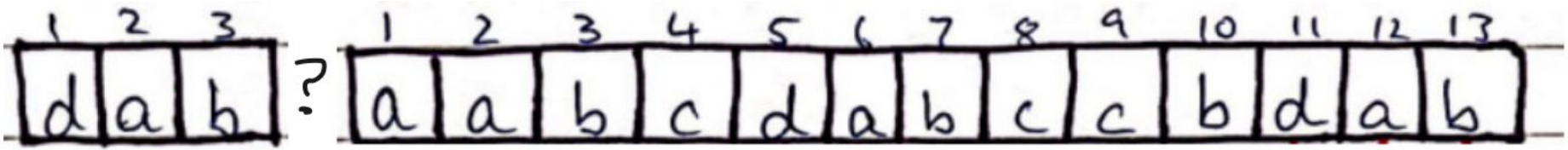
| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | 0 | 0 | 0 | 0 | 1 | 2 | 3 | 0 | 0 | 0 | 1 | 2 | 3 |

| | 1 | 2 | 3 |
|---|---|---|---|
| P: | d | a | b |

| | | | |
|---|---|---|---|
| B: | 0 | 0 | 0 |

ini

# Solucion KMP

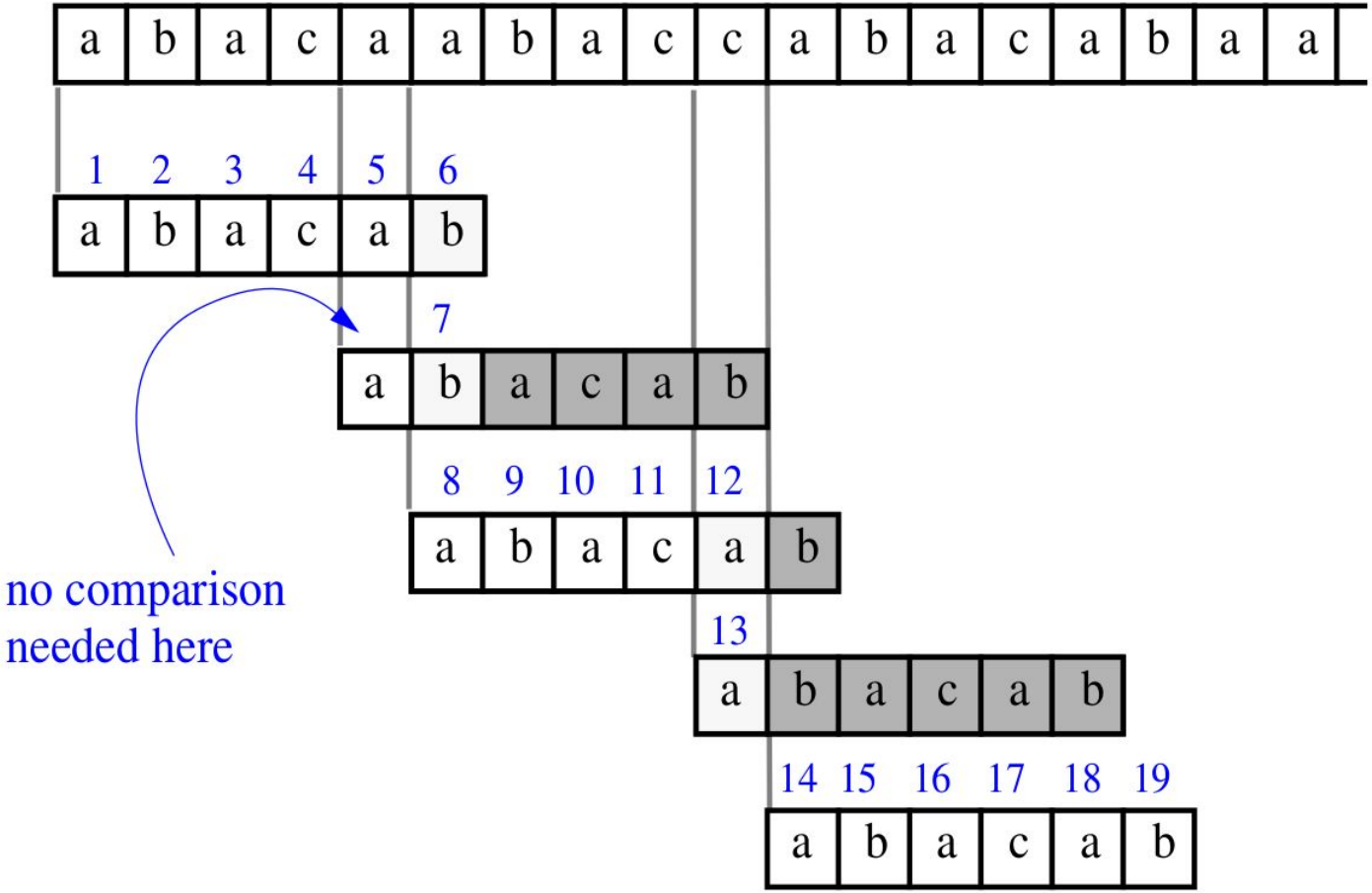


# Ejemplo KMP - String p

| | | | | | | |
|---|---|---|---|---|---|---|
| a | b | c | d | a | b | d |

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 2 | 0 |

# Solucion KMP



# Solucion: KMP

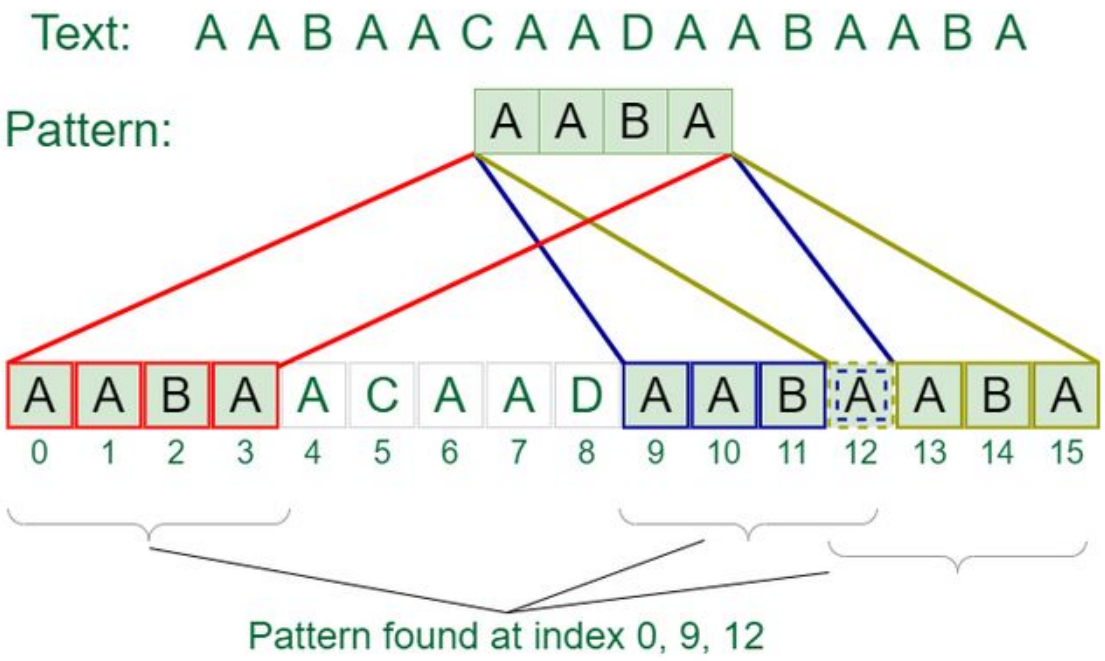
```
#define vector<int> vi

vi kmpp(string &s, int n) {
    vi dp(n, 0);
    int j= 0;
    for (int i = 1; i<n; i++) {
        while (j && s[i]!=s[j]) j = dp[j-1];
        if (s[i]==s[j]) dp[i] = ++j;
        else dp[i] = 0;
    }
    return dp;
}

vi kmp(string &t, string &p) {
    string q = p+"#"+t;
    vi v = kmpp(q, q.size());
    vi res;
    for (int i=0 ; i<q.size(); i++)
    if (v[i]==p.size()) {
        res.push_back(i-2*p.size()+1);
    }
    return res;
}
```

# Funcion Z

Otro algoritmo utilizado para el procesamiento de cadenas tenemos el algoritmo de funcion z, el cual nos permite conocer el prefijo mas grande del indice en el que este se encuentra, a diferencia del algoritmo KMP, el indice que almacena dicha informacion no es el ultimo que realiza ‘match’, sino el primero de estos.

| | |
|---|---|
| String | - a b c d e a a b d f a a b d f g |
| Pattern | - a a b b |
| New-String | - a a b b # a b c d e a a b b t a a b d f g |
| Z-array | - 0 1 0 0 0 1 0 0 0 0 4 1 0 0 0 3 1 0 0 0 0 |

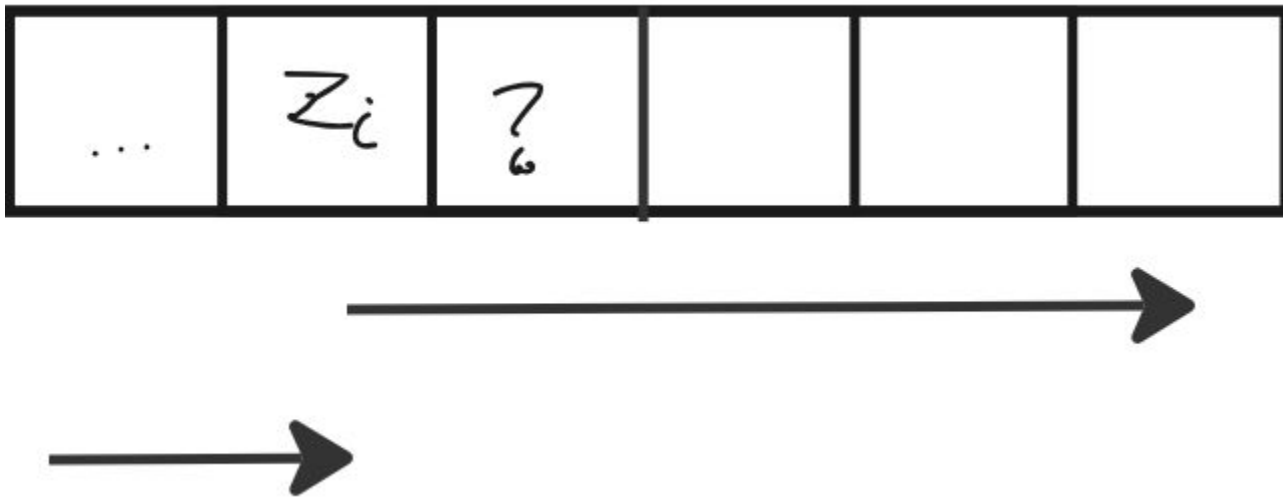
# Funcion Z


Text: A A B A A C A A D A A B A A B A
Pattern: A A B A
A A B A A C A A D A A B A A B A
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Pattern found at index 0, 9, 12

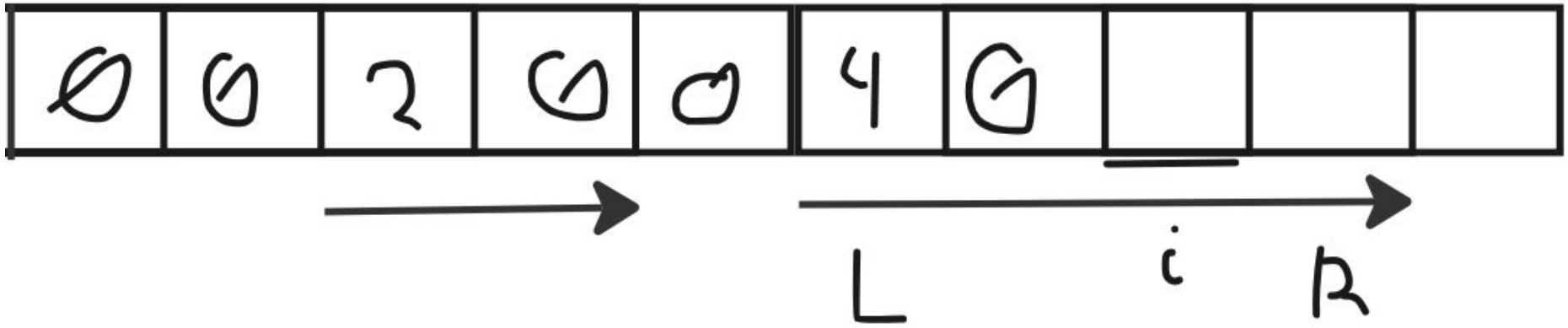
# Funcion Z - Algoritmo por fuerza bruta O(n^2)

```
vector<int> z_function_trivial(string s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1; i < n; i++) {
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
    }
    return z;
}
```

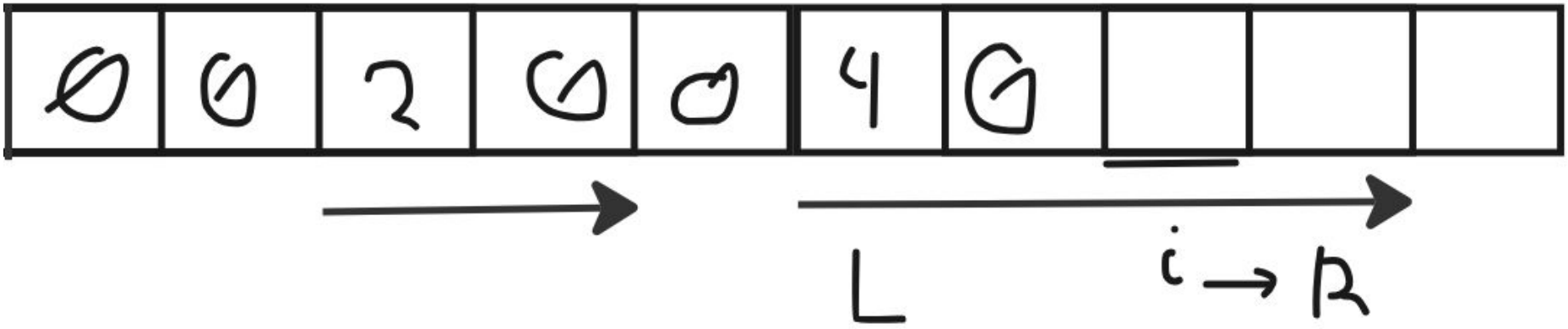
# Funcion Z - Optimizacion



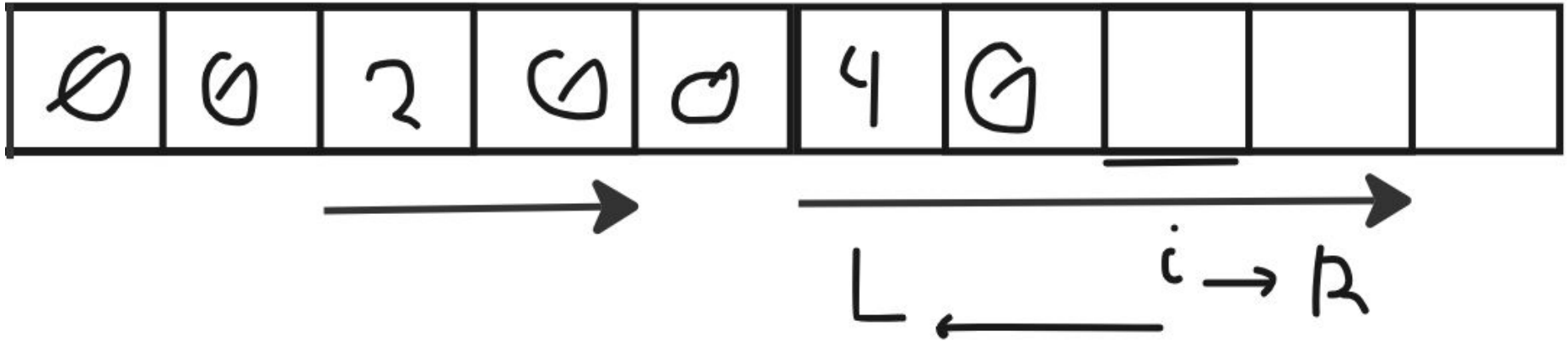
# Caso 1: Dentro de un prefijo anterior



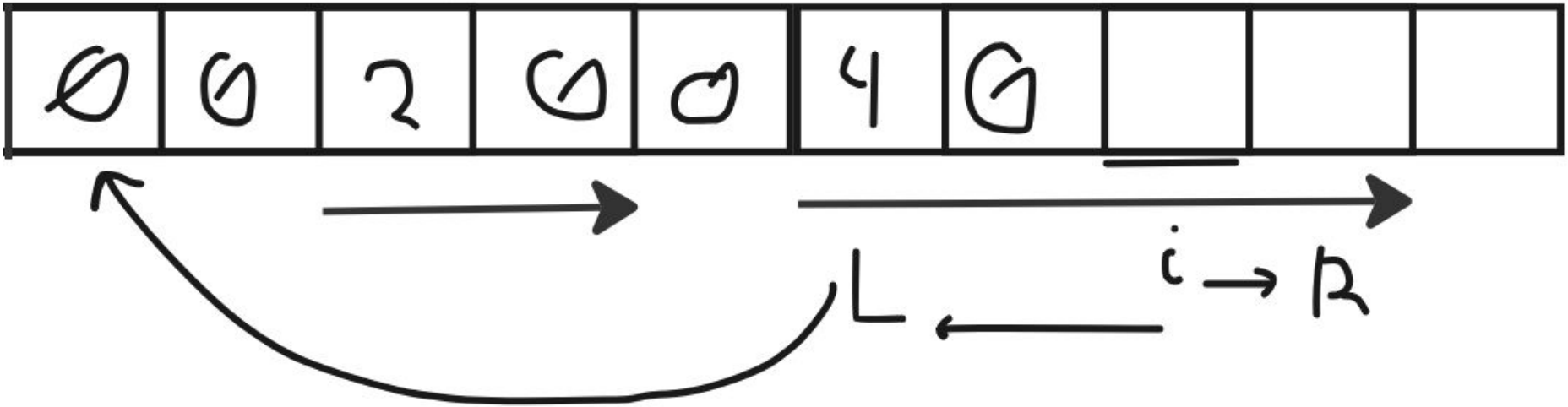
# Caso 1



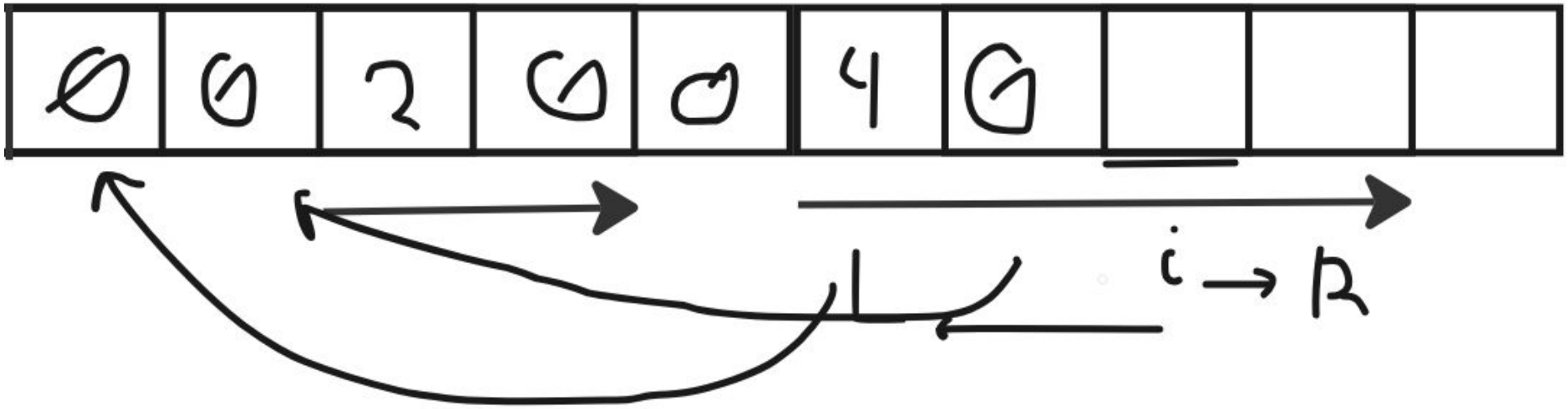
# Caso 1



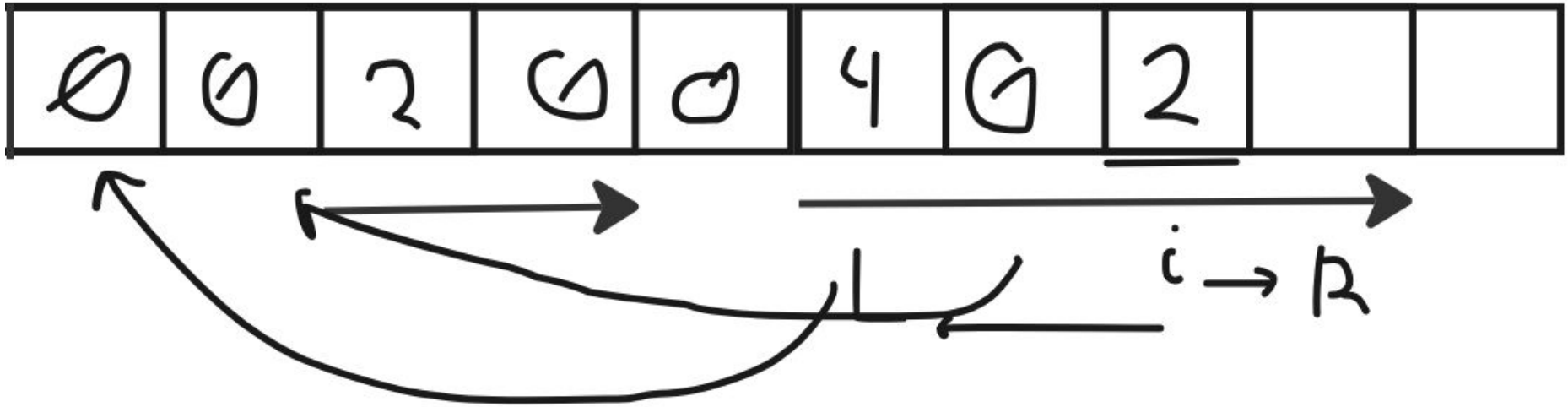
# Caso 1



# Caso 1:



# Caso 1



# Caso 2: No existe un prefijo anterior

Aplicamos fuerza bruta

```
vector<int> z_function_trivial(string s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1; i < n; i++) {
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
    }
    return z;
}
```

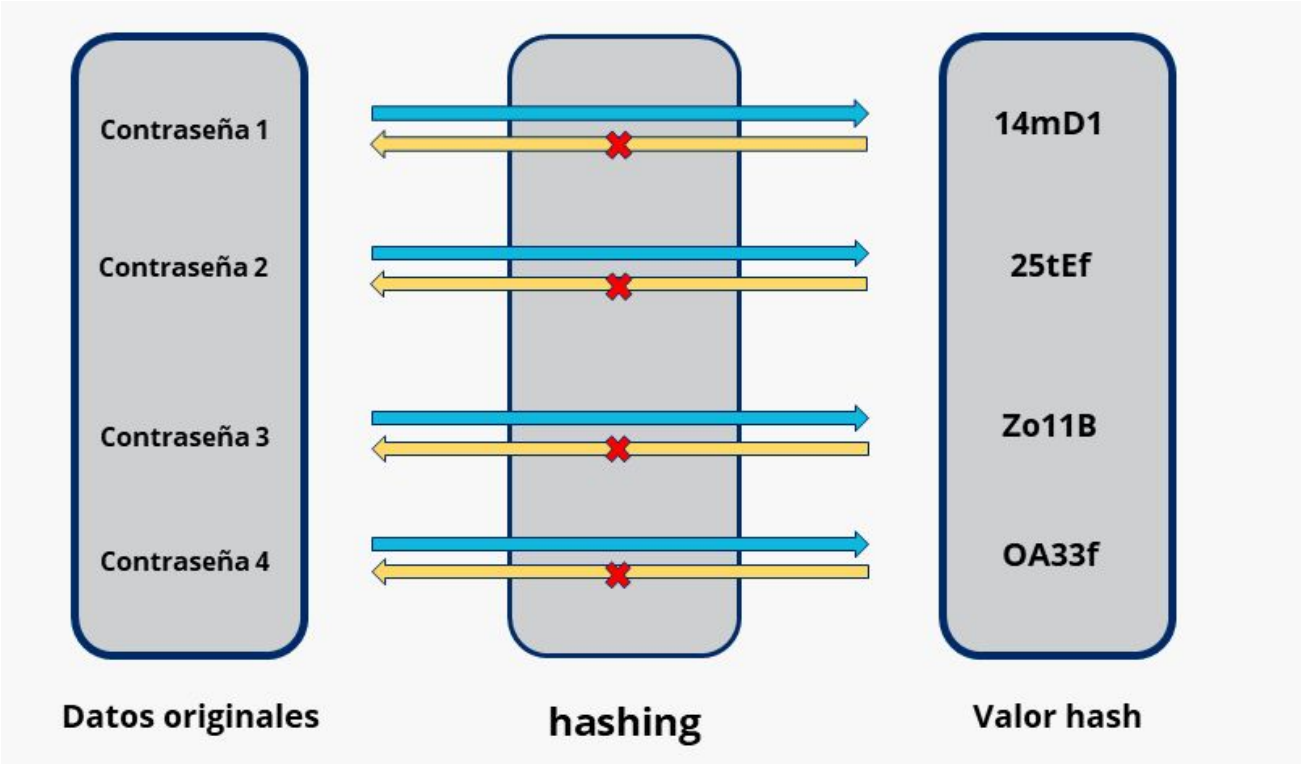
# Funcion Z

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

# Otra manera de solucion: Rabin - Karp

La idea central del algoritmo a diferencia de la versión *brute force* es usar una función *Hash* para comparar cada carácter, es decir transformar cada sub cadena de texto en un valor número (entero), eso es lo que hace el *hash*. Esto permite poder hacer comparación a través de su *hash* el cual necesita de hacer un preprocesamiento previo para el texto de largo *m*, así después se va desplazando en una ventana deslizante de *m*. Es importante elegir bien la función *hash* para evitar colisiones, porque cuando un texto da un valor v1, y el que se busca es v2, puede ser que sean igual pero el texto sea distinto, por eso se necesita una doble comparación (esto ocurre poco). También se necesita un número primo *q* y un valor para la base de la función *hash*. Esto se ha estudiado bastante en la literatura.

# Funcion hash



# Funcion hash

$$f(t_{i..i+|p|}) \rightarrow hash1$$

$$f(p) \rightarrow hash2$$

$$t_{i..i+|p|} = p \longrightarrow hash1 = hash2$$

$$hash1 = hash2 \longrightarrow t_{i..i+|p|} \stackrel{?}{=} p$$

# Funcion hash de una cadena

$$hash(s) = \sum_{i=0}^{|s|-1} (S_i * p^i) mod(M)$$

Donde:
**S** = es la cadena
**p** = es un numero, que debe ser al menos mas grande que el alfabeto que utilicemos y co-primo a M
**M** = es un numero muy grande para controlar el desbordamiento

# Funcion hash de una cadena

Consideraciones a la hora de elegir un p y M

$$\text{hash}(a, p, m) = (a_0 + a_1 \cdot p + a_2 \cdot p^2 + \ldots + a_{n-1} \cdot p^{n-1}) \bmod m$$

$$\max(a_i) < p < m$$

Recomiendo p = 31 y m = 1e9+7 para strings “pequeños”

# Colision de valores de hash

Paradoja de cumpleaños

**How many people must be there in a room to make the probability 100% that at-least two people in the room have same birthday?**

Answer: 367 (since there are 366 possible birthdays, including February 29).

Utilizando la serie de Taylor en la probabilidad de que al menos 2 personas cumplan el mismo dia:

$$n \approx \sqrt{2 \times 365 ln \left( \frac{1}{1-p(same)} \right)}$$

A la hora de elegir nuestros valores de hashing, considerar la siguiente formula:

$$p \approx 1 - \exp\left(-\frac{n^2}{2m}\right) \approx \frac{n^2}{2m}$$

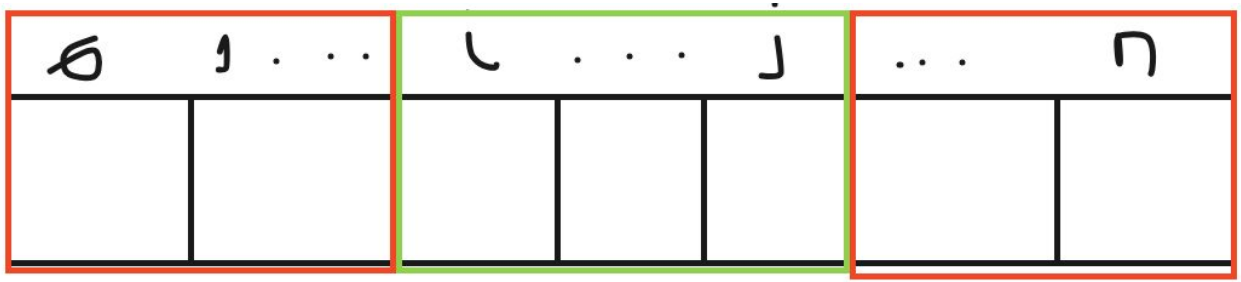
# Hash rodante / Hash polinomial

$$hash(s_{0,0}) = (S_0 * p^0) mod(M)$$

$$hash(s_{0,1}) = (S_0 * p^0 + S_1 * p^1) mod(M)$$

$$hash(s_{0,2}) = (S_0 * p^0 + S_1 * p^1 + S_2 * p^2) mod(M)$$

$$hash(s_{0,i}) = (hash(s_{0,i-1}) + S_i * p^i) mod(M)$$



$$hash(s_{i,j}) = ?$$

# Hash rodante / Hash polinomial

$$hash(s_{i,j}) = ((hash(s_{0,j}) - hash(s_{0,i-1}) + M) * p^{-i}) mod(M)$$

```
ll pow(ll b, ll e, ll m) {
        ll res = 1;
        for (; e; e>>=1, b = (b*b)%m)
        if (e&1) res = (res*b)%m;
        return res;
}
ll inv(ll b, ll e, ll m) {
        return pow(pow(b, e, m), m-2, m);
}
vi rollingHash(string &s, ll p, ll m) {
        ll n = s.size();
        vi v(n, 0);
        v[0] = (s[0])%m;
        for (ll i = 1; i<n; i++)
                v[i] = (v[i-1]+(s[i]*pow(p, i, m))%m)%m;
        return v;
}
ll fastHash(vi &v, ll p, ll m, ll i, ll j) {
        return (((v[j]-(i?v[i-1]: 0)+m)%m)*inv(p, i, m))%m;
}
```

# Solucion Rabin - Karp

Hash value of “AAAAA” is 37

Hash value of “AAAAH” is 100

# Solucion Rabin - Karp

1) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

2) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

3) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

4) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
AAAAH
37≠100 **1 comparison made**

# Solucion Rabin - Karp

N) AAAA

# Solucion Rabin - Karp

```
do
  if (hash_p == hash_t)
     brute force comparison of pattern
     and selected section of text
  hash_t = hash value of next section of
                text, one character over
while (end of text or
        brute force comparison == true)
```

# Solucion Rabin - Karp

```
#define vi vector<long long>
#define ll long long
const ll mod[2] = {1000000007,998244353};
const ll px[2] = {29, 31};

vector<int> kabinKarp(string &s, string &p) {
        vi ss[2], pp[2], ppx[2];
        for (ll i = 0; i<2; i++)
                ss[i] = rollingHash(s, px[i], mod[i]),
                pp[i] = rollingHash(p, px[i], mod[i]);

        vector<int> res;
        for (int i = 0; i+p.size()-1<s.size(); i++) {
                ll ok = 1;
                for (ll j = 0; j<2; j++) {
                    int fh = fastHash(ss[j], px[j], mod[j], i, i+p.size()-1)%mod[j];
                    ok &= (fh==pp[j].back());
                }

                if (ok) res.push_back(i+1);
        }
        return res;
}
```

# Comparación entre los algoritmos presentados

| Algoritmo | Complejidad temporal | Complejidad espacial |
|---|---|---|
| Brute Force | (n*m) | n+m |
| KMP | n+m | n+m |
| Function Z | n+m | n+m |
| Rabin Karp | n+m | n+m |

# Otro problema: Conteo de ocurrencia de prefijos

Este problema tienes 2 variaciones generales:

1. Dado un string s se quiere saber el numero de veces que aparece cada prefijo.
2. Dado un string s y otro t, se quiere saber el numero de veces que aparece un prefijo de t en el string s.

$$s[0 \ldots i]$$

# Solucion

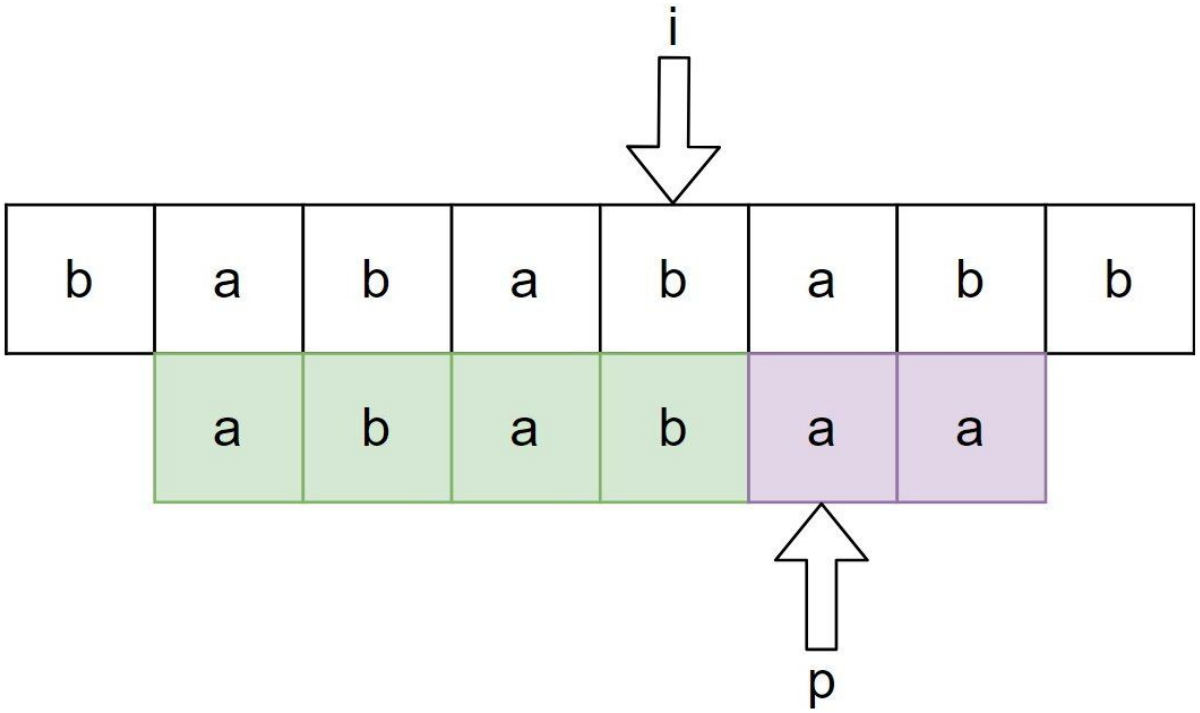
Tener las siguientes consideraciones, si tenemos el array KMP:

| | | | | | | |
|---|---|---|---|---|---|---|
| a | b | c | d | a | b | d |

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 2 | 0 |

# Solucion

Caracteristica de KMP:



# Solucion

```
vector<int> ans(n + 1);
for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n-1; i > 0; i--)
    ans[pi[i-1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++;
```